

Mỗi vị trí chứa cụm từ **replace me** tương ứng 01 hoặc NHIỀU hình ảnh, hãy thực hiện yêu cầu và dán 01/các ảnh chụp toàn màn hình kết quả đạt được và mô tả tóm tắt ý nghĩa dưới mỗi hình ảnh. Thay "(Thực hiện theo yêu cầu trên)" bằng lời giải thích cho hình chụp đó (thường là nghĩa tiếng việt dịch được theo yêu cầu tiếng anh), hoặc giải thích ngắn gọn hơn theo ý sinh viên, không giải thích **-25%** điểm câu đó

1. Critical Section (5 Points):

Khi giải quyết bài toán miền găng chú ý 4 điều kiện sau:

- (1) Không có hai tiến trình cùng ở trong miền găng cùng lúc.
- (2) Không có giả thiết về tốc độ của các tiến trình, cũng như về số lượng bộ xử lý.
- (3) Một tiến trình bên ngoài miền găng không được ngăn cản các tiến trình khác vào miền găng.
- (4) Không có tiến trình nào phải chờ vô hạn để được vào miền găng.

Synchronization là gì ?

Các tiến trình cần hợp tác với nhau để hoàn thành công việc.

Các giải pháp đồng bộ: Chia làm 2 nhóm tùy theo cách tiếp cận trong xử lý tiến trình bị khóa:

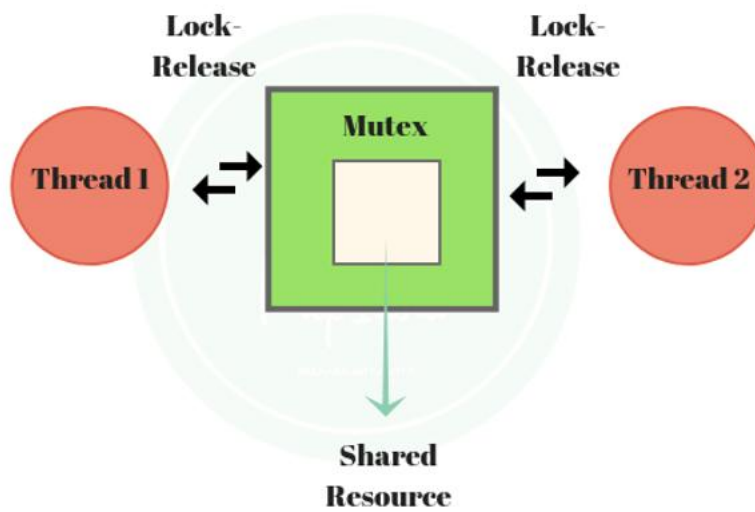
- Nhóm giải pháp “busy waiting”
 - Các giải pháp phần mềm
 - Các giải pháp phần cứng
- Nhóm giải pháp “sleep and wakeup”
 - Semaphore
 - Monitors
 - Trao đổi thông điệp

Mutex là gì? Mutual exclusion

Mutex là một cờ hiệu, đối với hệ thống Mutex là một đối tượng mang hai trạng thái đang được sử dụng và chưa sử dụng (trạng thái sẵn sàng). Khi mutex bật, một luồng sẽ bước vào sử dụng tài nguyên và khóa mutex. Luồng khác sẽ không sử dụng được tài nguyên cho đến khi luồng hiện tại mở khóa mutex.



Mutex in Operating System



1.1 Cho đoạn mã sau

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>

#define MAX_THREAD      2

pthread_t tid[MAX_THREAD];
/*Counter la bien toan cuc duoc 2 thread su dung*/
int counter;

void *threadFunc(void *argv)
{
    counter += 1;
    printf("Thread %d has started\n", counter);

    sleep(1);
    printf("Thread %d has finished\n", counter);

    pthread_exit(NULL);
}

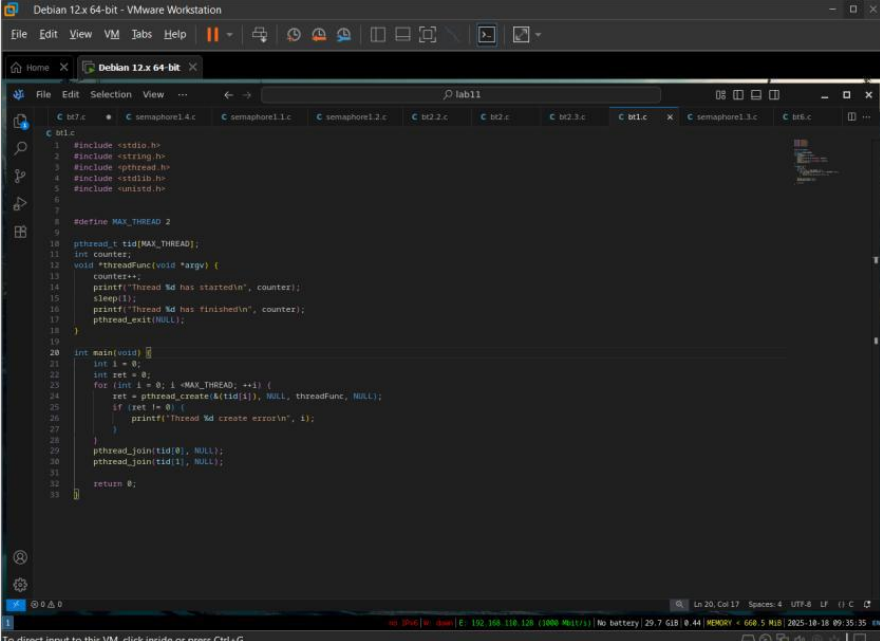
int main(void)
{
    int i = 0;
    int ret = 0;

    for (i = 0; i < MAX_THREAD; i++)
    {
        ret = pthread_create(&(tid[i]), NULL, threadFunc, NULL);
        if (ret != 0)
        {
            printf("Thread [%d] created error\n", i);
        }
    }

    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);

    return 0;
}
```

a. Tạo và lưu lại đoạn mã với tên là bt1.c



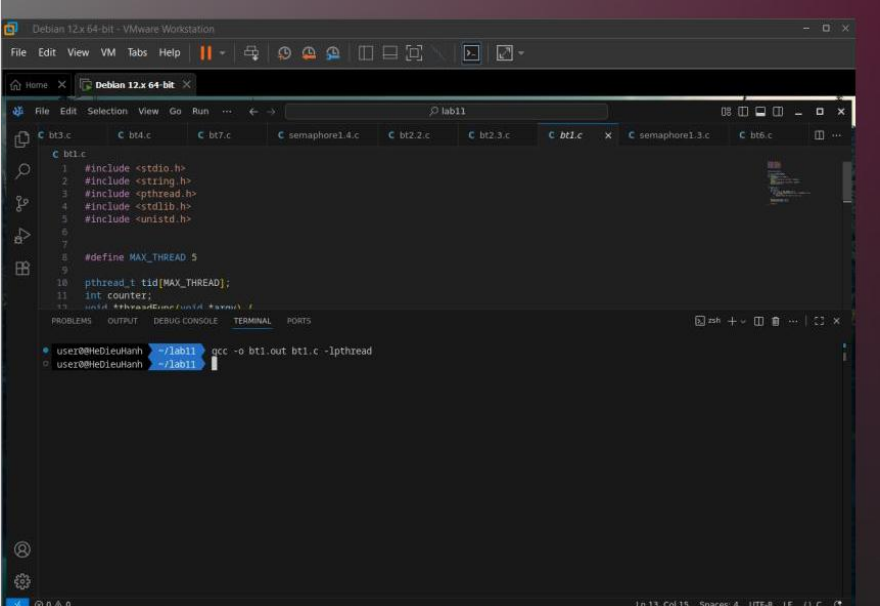
```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7
8 #define MAX_THREAD 2
9
10 pthread_t tid[MAX_THREAD];
11 int counter;
12 void *threadfunc(void *argv) {
13     counter++;
14     printf("Thread %d has started\n", counter);
15     sleep(1);
16     printf("Thread %d has finished\n", counter);
17     pthread_exit(NULL);
18 }
19
20 int main(void) {
21     int i = 0;
22     int ret = 0;
23     for (int i = 0; i < MAX_THREAD; ++i) {
24         ret = pthread_create(&tid[i], NULL, threadfunc, NULL);
25         if (ret != 0) {
26             printf("Thread %d create error\n", i);
27         }
28     }
29     pthread_join(tid[0], NULL);
30     pthread_join(tid[1], NULL);
31     return 0;
32 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là code của chương trình.

b. Sau đó compile đoạn mã trên với lệnh **gcc -o bt1.out bt1.c -lpthread**.



```
user@HeDeuAnh: ~/lab11$ gcc -o bt1.out bt1.c -lpthread
user@HeDeuAnh: ~/lab11$
```

ID: 2324802010295

NAME: Pham Khanh Minh

The screenshot shows a VMware Workstation interface with a Debian 12 VM. The main window displays a C program in a text editor. The program includes `<stdio.h>`, `<string.h>`, `<pthread.h>`, and `<unistd.h>`. It defines `MAX_THREAD` as 2 and creates two threads. Each thread increments a global `counter` and prints its status. The terminal output shows the execution results, where Thread 1 has started and finished, and Thread 2 has started and finished. The output is as follows:

```
user@debian: ~/lab11$ gcc -o bt1.out bt1.c -lpthread
user@debian: ~/lab11$ ./bt1.out
Thread 1 has started
Thread 2 has started
Thread 2 has finished
Thread 1 has finished
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là khi chạy chương trình.

c. Cho biết CS trong đoạn chương trình trên.

Trong `threadFunc`

`counter++`; `printf("Thread %d has started\n", counter);`

`sleep(1);`

`printf("Thread %d has finished\n", counter);`

`counter++` là CS

Vì nhiều luồng cùng truy cập và thay đổi biến toàn cục `counter` mà không được bảo vệ.

d. Nhận xét kết quả sau khi thực thi file `bt1.out`

Nhiều Thread hoạt động đồng thời gây ra lỗi biến `counter` dẫn đến

Thread 1 chưa has finished còn thread 2 thì lại has finished tận 2 lần

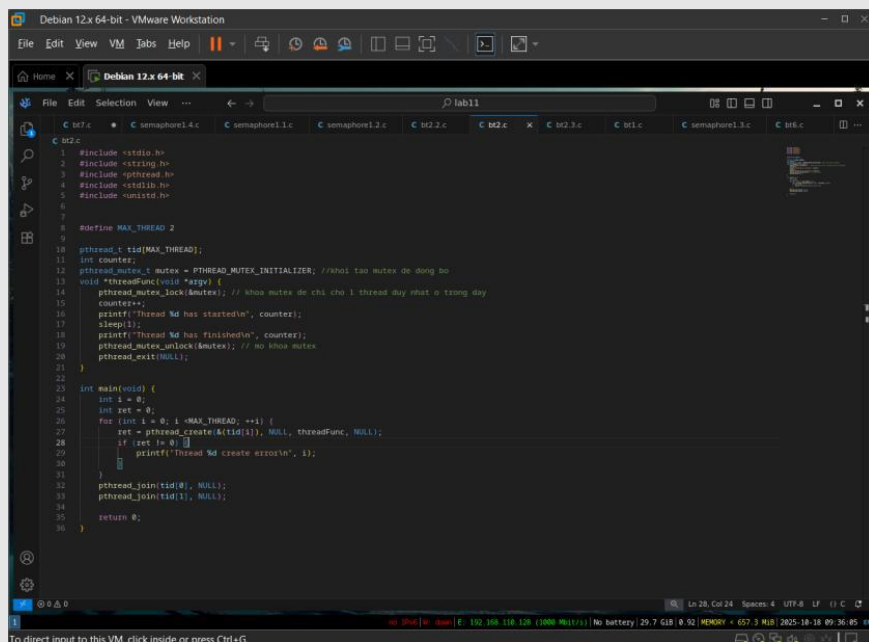
- 1.2 Cho đoạn mã theo yêu cầu 1.1, save as bt1.c thành bt2.c, sau đó thêm các dòng sau đây vào bt2.c:

```
int counter;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; //khai bao mutex
```

```
void *threadFunc(void *argv)
{
    pthread_mutex_lock(&mutex);
    counter += 1;
    printf("Thread %d has started\n", counter);
```

```
    sleep(1);
    printf("Thread %d has finished\n", counter);
    pthread_mutex_unlock(&mutex);
}
```

- a. Giải thích đoạn mã `pthread_mutex_lock(&mutex)` và `pthread_mutex_unlock(&mutex)` bằng ký hiệu // cùng dòng



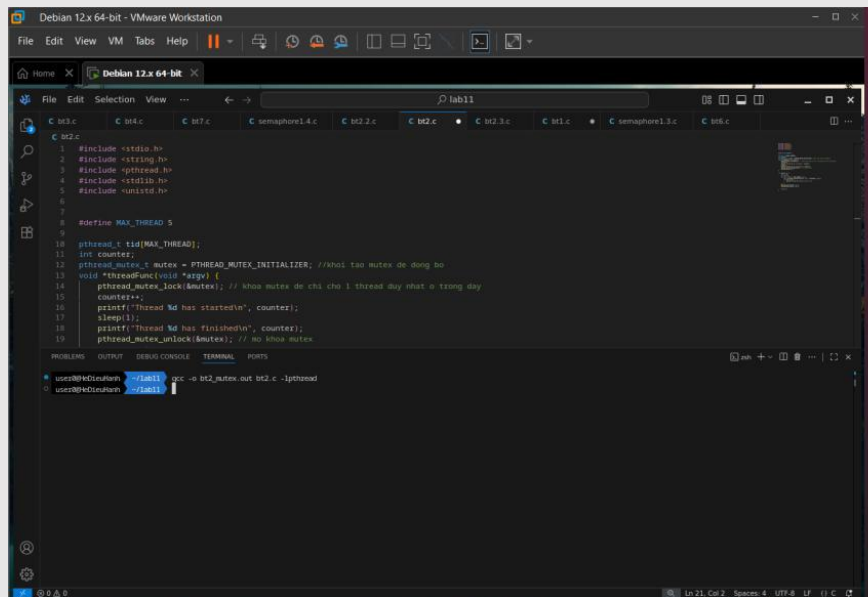
```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4 #include <unistd.h>
5 #include <unistd.h>
6
7
8 #define MAX_THREAD 2
9
10 pthread_t tid[MAX_THREAD];
11 int counter;
12 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; //khai tao mutex de dong bo
13 void *threadFunc(void *argv) {
14     pthread_mutex_lock(&mutex); // khóa mutex để chỉ cho 1 thread duy nhất ở trong đây
15     counter++;
16     printf("Thread %d has started\n", counter);
17     sleep(1);
18     printf("Thread %d has finished\n", counter);
19     pthread_mutex_unlock(&mutex); // bỏ khóa mutex
20     pthread_exit(NULL);
21 }
22
23 int main(void) {
24     int i = 0;
25     int ret = 0;
26     for (int i = 0; i < MAX_THREAD; ++i) {
27         ret = pthread_create(&tid[i], NULL, threadFunc, NULL);
28         if (ret != 0) {
29             printf("Thread %d create error\n", i);
30         }
31     }
32     pthread_join(tid[0], NULL);
33     pthread_join(tid[1], NULL);
34     return 0;
35 }
```

ID: 2324802010295

NAME: Phạm Khanh Minh

Màn hình trên là code của chương trình.

- b. Compile bt2.c thành file **bt2_mutex.out**



```
#include <stdio.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>

#define MAX_THREAD 5

pthread_t tid[MAX_THREAD];
int counter;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER; //khởi tạo mutex để đồng bộ

void *threadFunc(void *argv) {
    pthread_mutex_lock(&mutex); // khóa mutex để chỉ cho 1 thread duy nhất vào trong đây
    counter++;
    printf("Thread %d has started\n", counter);
    sleep(1);
    printf("Thread %d has finished\n", counter);
    pthread_mutex_unlock(&mutex); // mở khóa mutex
}
```

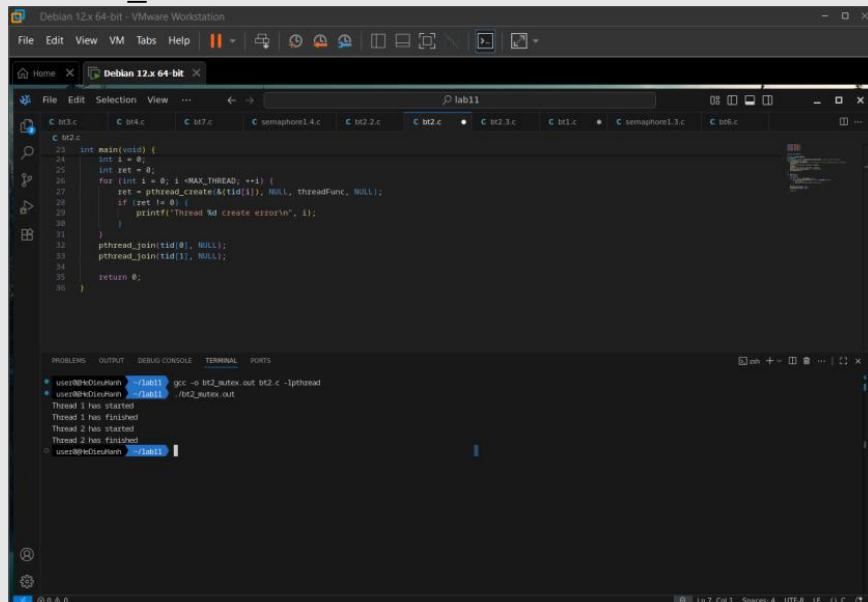
```
gcc -o bt2_mutex.out bt2.c -lpthread
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là khi biên dịch chương trình.

c. Run bt2_mutex.out



```
int main(void) {
    int i = 0;
    int ret = 0;
    for (int i = 0; i < MAX_THREAD; ++i) {
        ret = pthread_create(&tid[i], NULL, threadFunc, NULL);
        if (ret != 0) {
            printf("Thread %d create error\n", i);
        }
    }
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
    return 0;
}
```

```
gcc -o bt2_mutex.out bt2.c -lpthread
```

```
Thread 1 has started
Thread 1 has finished
Thread 2 has started
Thread 2 has finished
```

ID: 2324802010295

NAME: Pham Khanh Minh

d. So sánh kết quả với bt1.out của yêu cầu 1.1

So với bt1.out thì nó đã ổn định hơn rất nhiều. Do có mutex mỗi thread chỉ đi vào ở 1 thời điểm, kết quả chính xác và đúng thứ tự.

1.3 Cho đoạn mã sau:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

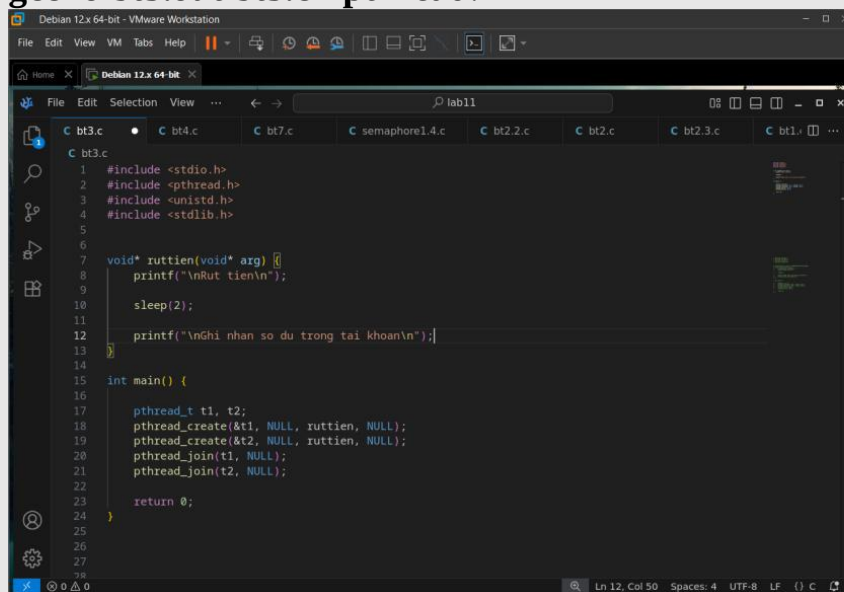
void* ruttien(void* arg)
{
    printf("\nRut tien\n");

    sleep(2);

    printf("\nGhi nhan so du trong tai khoan\n");
}

int main()
{
    pthread_t t1,t2;
    pthread_create(&t1,NULL,ruttien,NULL);
    pthread_create(&t2,NULL,ruttien,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    return 0;
}
```

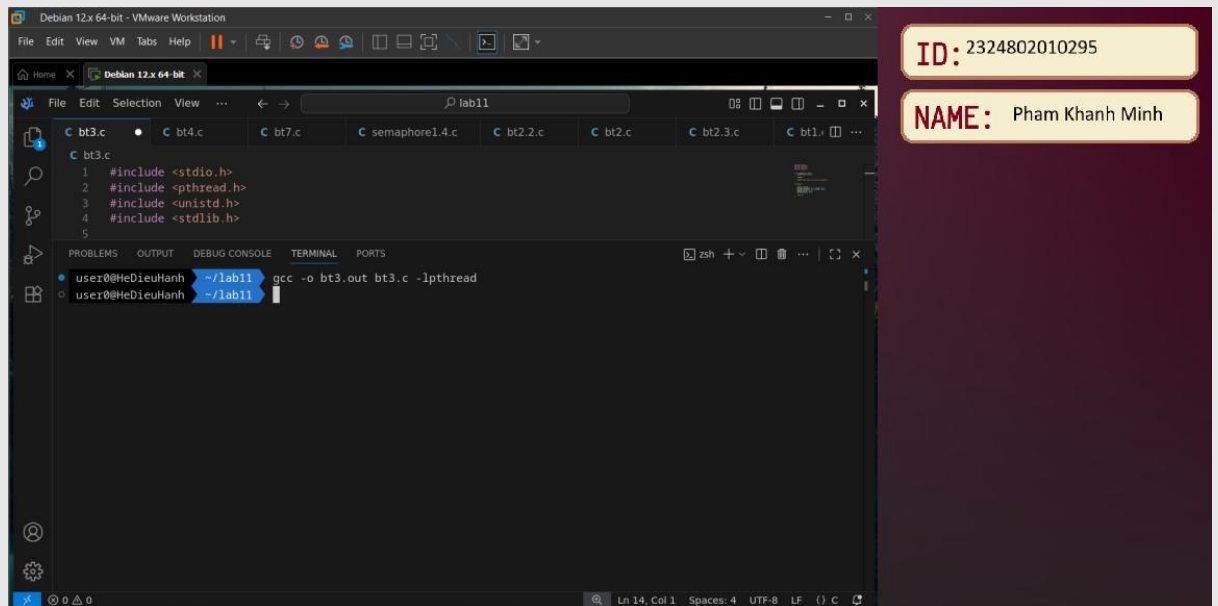
- a. Lưu lại đoạn mã với tên là bt3.c, Sau đó compile đoạn mã trên với lệnh **gcc -o bt3.out bt3.c -lpthread**.



ID: 2324802010295

NAME: Phạm Khanh Minh

Màn hình trên là code của chương trình.



Màn hình trên là biên dịch chương trình.

b. Cho biết Critical Section trong đoạn chương trình trên.

Trong hàm ruttien

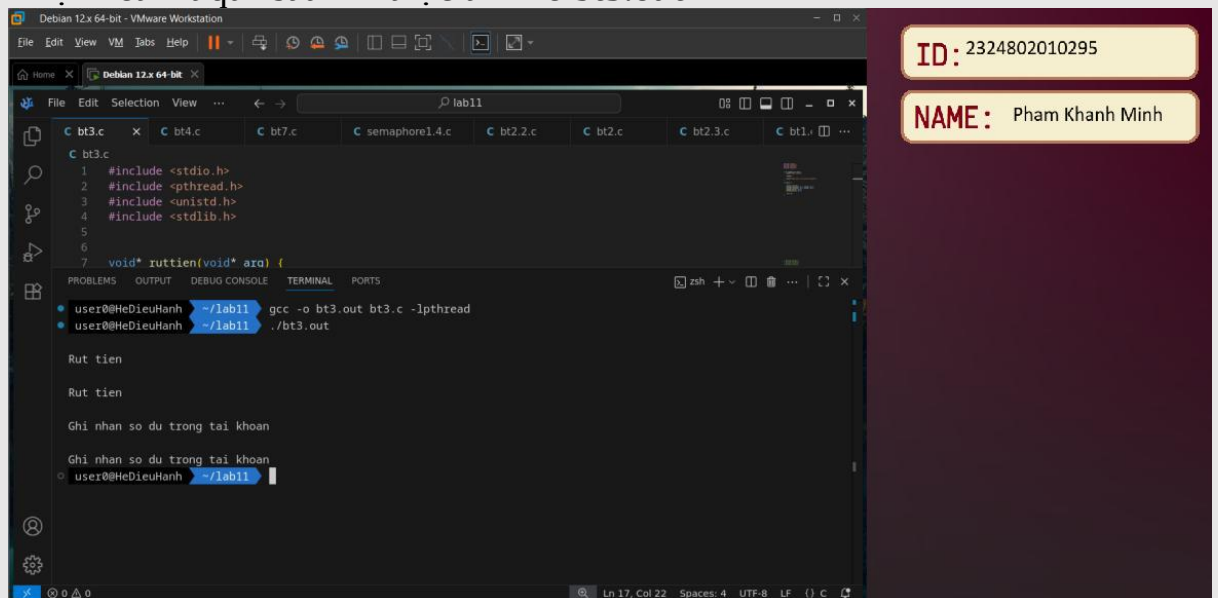
printf("\nRut tien\n");

sleep(2);

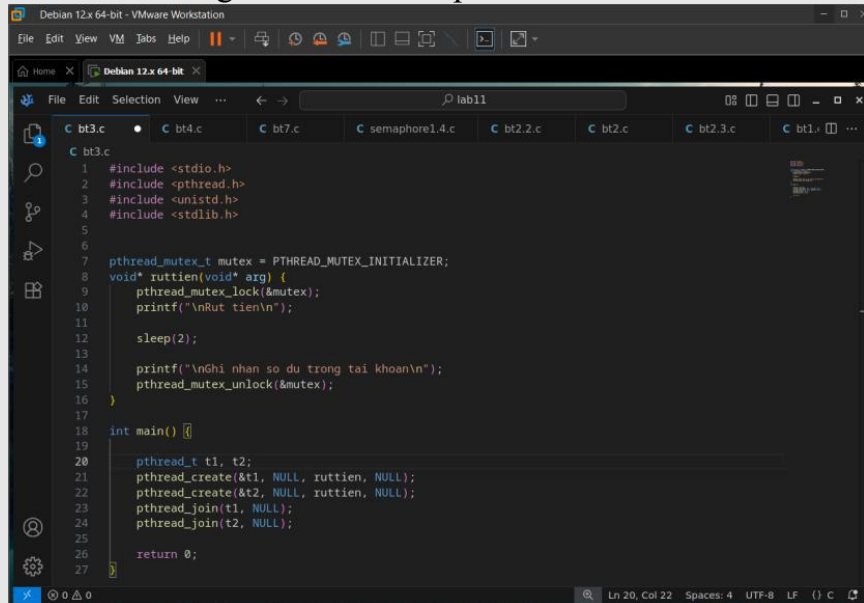
printf("\nGhi Nhan so du trong tai khoan\n");

Cả hai thread có thể cùng thực hiện các lệnh printf và sleep, gây chồng chéo, lộn xộn thông tin.

c. Nhận xét kết quả sau khi thực thi file **bt3.out**



d. Sử dụng mutex để khắc phục lỗi của bt3.c

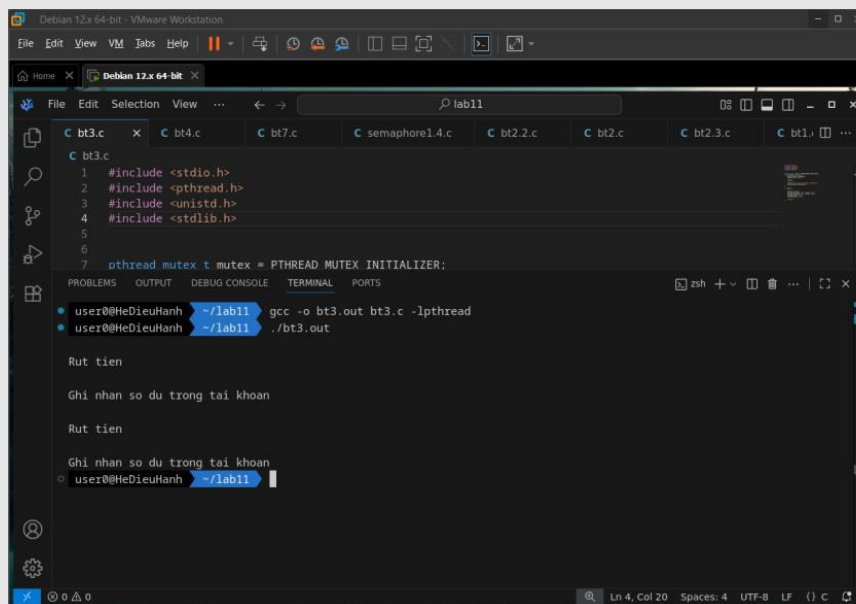


```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6
7 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
8 void* ruttien(void* arg) {
9     pthread_mutex_lock(&mutex);
10    printf("Rut tien\n");
11
12    sleep(2);
13
14    printf("Ghi nhan so du trong tai khoan\n");
15    pthread_mutex_unlock(&mutex);
16 }
17
18
19 int main() {
20
21    pthread_t t1, t2;
22    pthread_create(&t1, NULL, ruttien, NULL);
23    pthread_create(&t2, NULL, ruttien, NULL);
24    pthread_join(t1, NULL);
25    pthread_join(t2, NULL);
26
27    return 0;
28 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là đoạn code chương trình sau khi thêm mutex



```
user@HeDieuHanh: ~/lab11$ gcc -o bt3.out bt3.c -lpthread
user@HeDieuHanh: ~/lab11$ ./bt3.out

Rut tien

Ghi nhan so du trong tai khoan

Rut tien

Ghi nhan so du trong tai khoan
user@HeDieuHanh: ~/lab11$
```

ID: 2324802010295

NAME: Pham Khanh Minh

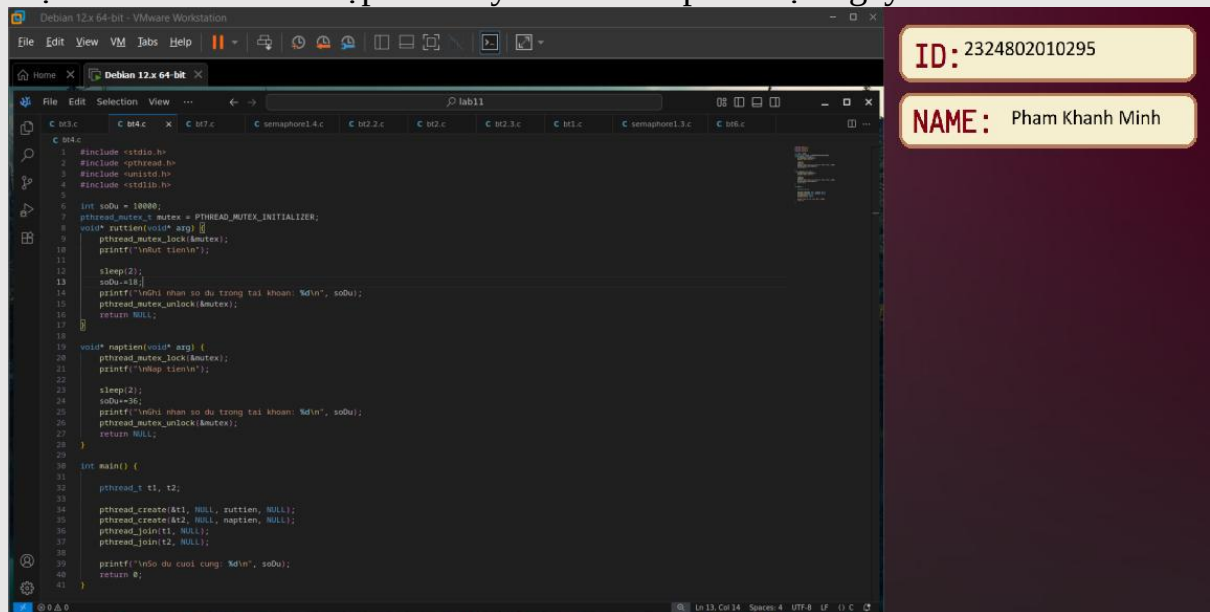
Màn hình trên là kết quả chương trình sau khi thêm mutex.

Lúc này sau khi rút tiền thì sẽ bắt đầu kiểm tra số dư rồi mới bắt đầu rút tiếp

1.4 Cho hai thread xử lý công việc rút tiền và nạp tiền được mô tả như sau:

```
void* ruttien(void* arg)
{
    printf("\nRut tien\n");
    sleep(2);
    printf("\nGhi nhan so du trong tai khoan\n");
}
void* naptien(void* arg)
{
    printf("\nNap tien\n");
    sleep(2);
    printf("\nGhi nhan so du trong tai khoan\n");
}
```

Hãy tạo 2 thread xử lý 2 luồng công việc trên và thực hiện đồng bộ hóa các công việc trên sau cho khi nạp tiền hay rút tiền thì phải hiện ngay số dư của tài khoản.



The screenshot shows a C program in a terminal window. The program defines two functions, `ruttien` and `naptien`, which simulate withdrawing and depositing money. It then creates two threads, `t1` and `t2`, one for each function. A mutex is used to ensure that the threads execute sequentially, preventing race conditions. The output shows the threads running in order, with the balance being updated correctly after each operation.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>

int soDu = 10000;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* ruttien(void* arg) {
    pthread_mutex_lock(&mutex);
    printf("\nRut tien\n");
    sleep(2);
    soDu -= 10;
    printf("\nGhi nhan so du trong tai khoan: %d\n", soDu);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

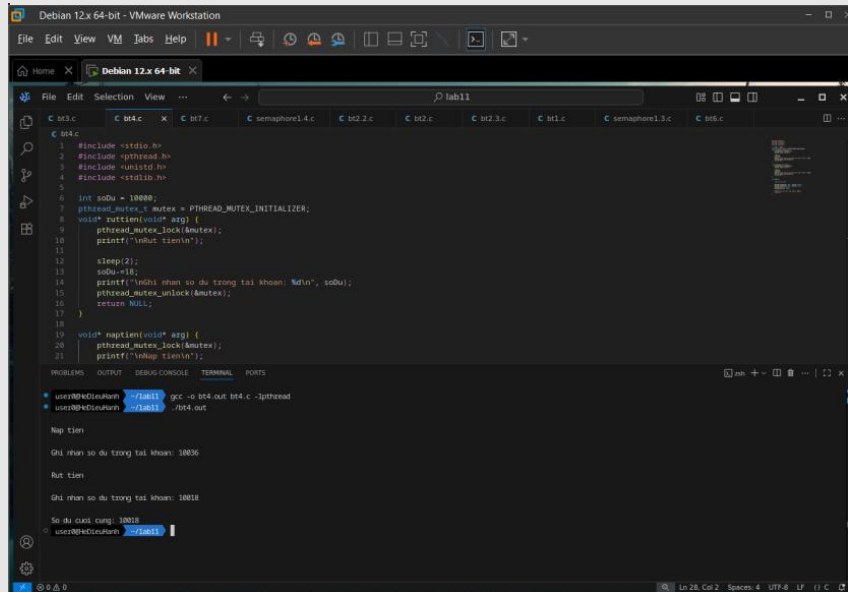
void* naptien(void* arg) {
    pthread_mutex_lock(&mutex);
    printf("\nNap tien\n");
    sleep(2);
    soDu += 10;
    printf("\nGhi nhan so du trong tai khoan: %d\n", soDu);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, ruttien, NULL);
    pthread_create(&t2, NULL, naptien, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("\nSo du cuoi cung: %d\n", soDu);
    return 0;
}
```

ID: 2324802010295

NAME: Phạm Khanh Minh

1. Màn hình trên là đoạn code chương trình xử lý 2 luồng `ruttien` và `naptien`.



```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 int sobu = 10000;
7 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
8 void* ruttien(void* arg) {
9     pthread_mutex_lock(&mutex);
10    printf("inrut tien\n");
11
12    sleep(2);
13    sobu -= 10;
14    printf("inrut nhan so du trong tai khoan: %d\n", sobu);
15    pthread_mutex_unlock(&mutex);
16    return NULL;
17 }
18
19 void* naptien(void* arg) {
20    pthread_mutex_lock(&mutex);
21    printf("venhap tien\n");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

user@kali:~/lab11\$ gcc -o b04.out b04.c -lpthread

user@kali:~/lab11\$./b04.out

Nạp tiền

Ghi nhận số dư trong tài khoản: 10000

Rút tiền

Ghi nhận số dư trong tài khoản: 10010

Số dư cuối cùng: 10010

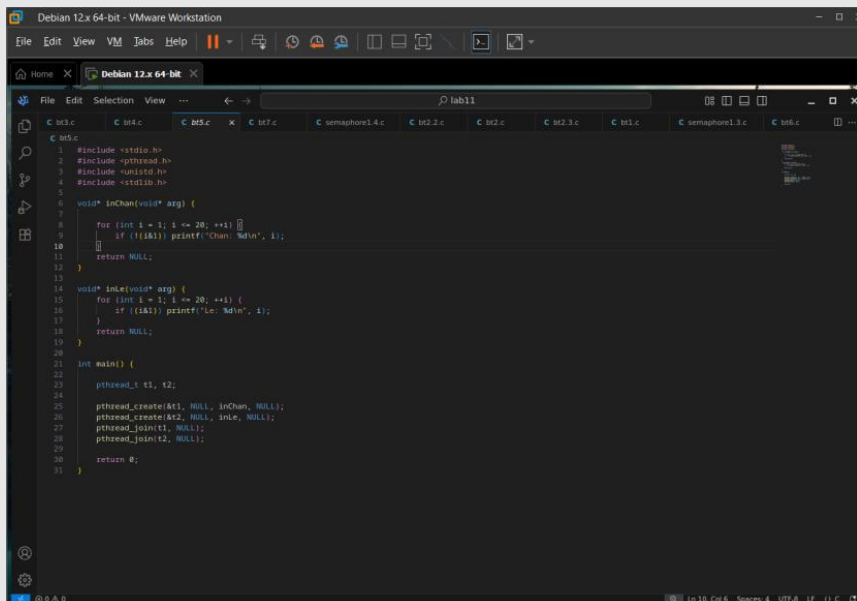
ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả của chương trình.

1.5 Viết chương trình tạo 2 thread xử lý 2 luồng công việc:

- Thread 1: in ra các số chẵn từ 1 tới 20.
- Thread 2: in ra các số lẻ từ 1 tới 20.



```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 void* inChan(void* arg) {
7     for (int i = 1; i <= 20; ++i) {
8         if ((i%2) == 0) printf("Chan %d\n", i);
9     }
10    return NULL;
11 }
12
13 void* inLe(void* arg) {
14     for (int i = 1; i <= 20; ++i) {
15         if ((i%2) == 1) printf("Le %d\n", i);
16     }
17    return NULL;
18 }
19
20 int main() {
21
22
23    pthread_t t1, t2;
24
25    pthread_create(&t1, NULL, inChan, NULL);
26    pthread_create(&t2, NULL, inLe, NULL);
27    pthread_join(t1, NULL);
28    pthread_join(t2, NULL);
29
30    return 0;
31 }
```

user@kali:~/lab11\$ gcc -o b05.out b05.c -lpthread

user@kali:~/lab11\$./b05.out

Chan 2

Le 1

Chan 4

Le 3

Chan 6

Le 5

Chan 8

Le 7

Chan 10

Le 9

Chan 12

Le 11

Chan 14

Le 13

Chan 16

Le 15

Chan 18

Le 17

Chan 20

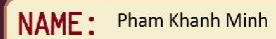
Le 19

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code chương trình xử lý 2 luồng :

- Thread 1: in ra các số chẵn từ 1 tới 20.
- Thread 2: in ra các số lẻ từ 1 tới 20.

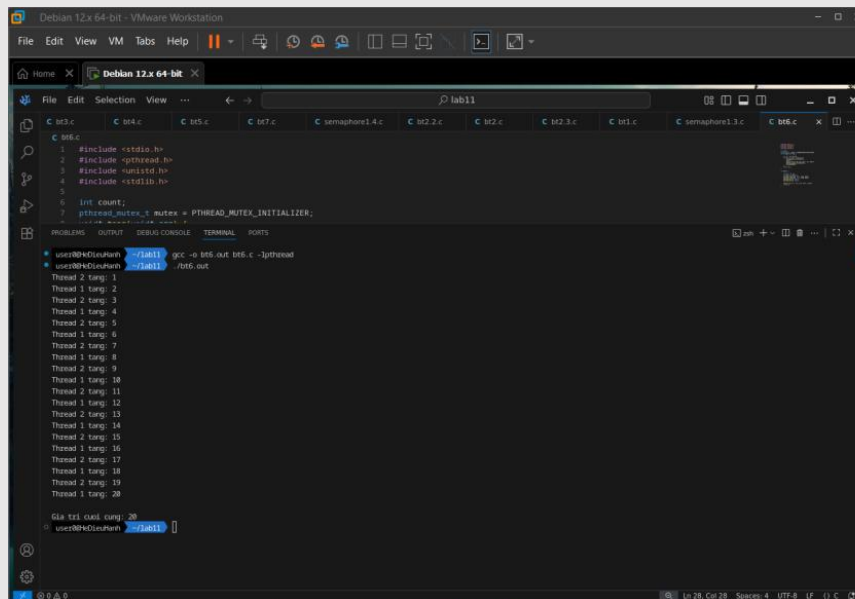


Màn hình trên là kết quả chương trình.
Đoạn code xử lý 1 cách tuần tự lẽ xong rồi tới chẵn.

1.6 Viết chương trình tạo 2 thread, cả 2 thread lần lượt tăng biến đó lên 1 đơn vị, sử dụng mutex để đồng bộ việc ghi vào biến đó.



Code chương trình xử lý 2 luồng 2 thread lần lượt tăng biến đó lên 1 đơn vị có sử dụng mutex để đồng bộ.



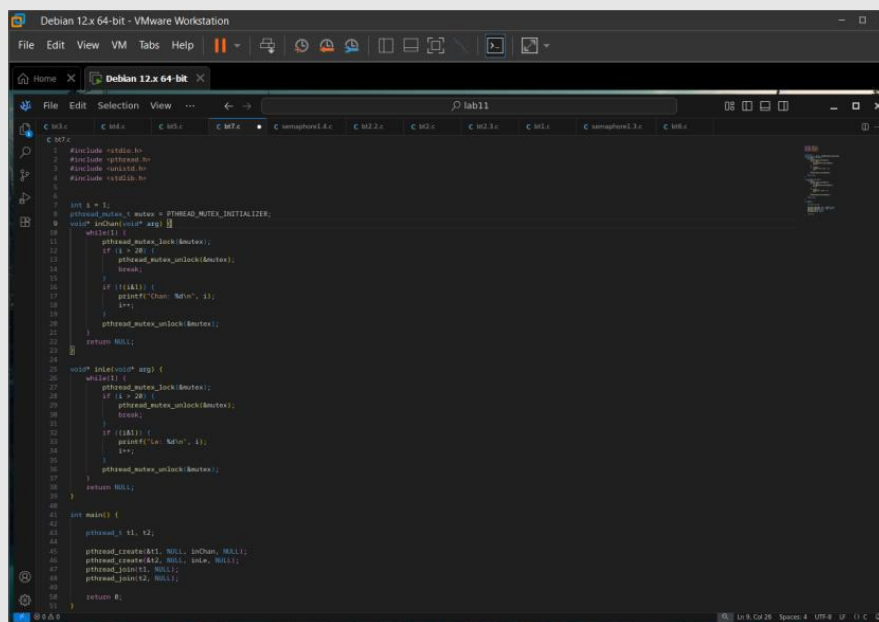
```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 int count;
7 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
8
9 void* thread1(void* arg) {
10     while(1) {
11         pthread_mutex_lock(&mutex);
12         if (count < 20) {
13             printf("Thread 1: %d\n", count);
14             count++;
15             pthread_mutex_unlock(&mutex);
16         }
17     }
18 }
19
20 void* thread2(void* arg) {
21     while(1) {
22         pthread_mutex_lock(&mutex);
23         if (count < 20) {
24             printf("Thread 2: %d\n", count);
25             count++;
26             pthread_mutex_unlock(&mutex);
27         }
28     }
29 }
30
31 int main() {
32     pthread_t t1, t2;
33     pthread_create(&t1, NULL, thread1, NULL);
34     pthread_create(&t2, NULL, thread2, NULL);
35     pthread_join(t1, NULL);
36     pthread_join(t2, NULL);
37     return 0;
38 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả chương trình.
Kết quả chạy đồng bộ giữa 2 luồng 1 và 2.

1.7 Hãy sử dụng mutex cho bt1.5 để màn hình in các số theo thứ tự từ 1 tới 20.

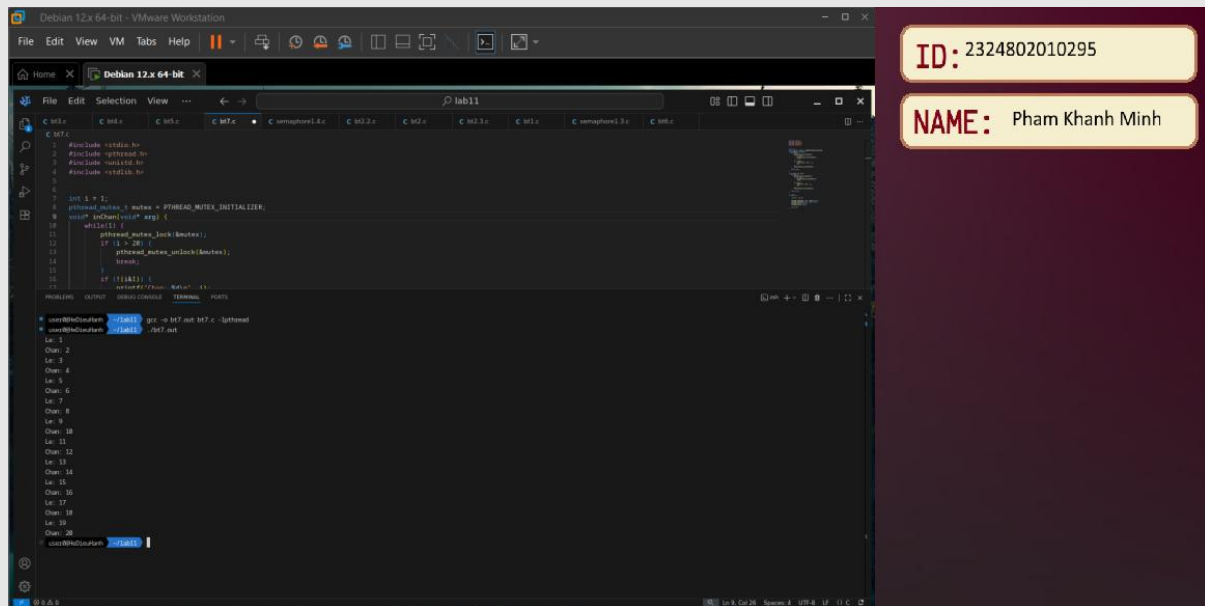


```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 int count = 1;
7 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
8
9 void* thread1(void* arg) {
10     while(1) {
11         pthread_mutex_lock(&mutex);
12         if (count < 20) {
13             printf("Thread 1: %d\n", count);
14             count++;
15             pthread_mutex_unlock(&mutex);
16         }
17     }
18 }
19
20 void* thread2(void* arg) {
21     while(1) {
22         pthread_mutex_lock(&mutex);
23         if (count < 20) {
24             printf("Thread 2: %d\n", count);
25             count++;
26             pthread_mutex_unlock(&mutex);
27         }
28     }
29 }
30
31 int main() {
32     pthread_t t1, t2;
33     pthread_create(&t1, NULL, thread1, NULL);
34     pthread_create(&t2, NULL, thread2, NULL);
35     pthread_join(t1, NULL);
36     pthread_join(t2, NULL);
37     return 0;
38 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code của chương trình sử dụng mutex cho bt1.5 để màn hình in các số theo thứ tự từ 1 tới 20.



Màn hình trên là kết quả chương trình
2 luồng chặn lẻ được chạy đồng bộ thay vì tuần tự như bt1.5

1. Semaphore (5 Points):

- Semaphore được sử dụng để đồng bộ các tiến trình và tiểu trình. Semaphore được kết hợp với hàng đợi thông điệp và bộ nhớ dùng chung trong các IPC (Inter-Process Communication) cơ sở trong các hệ thống UNIX.
 - Semaphore là một cờ hiệu tương tự MUTEX, nếu một tiểu trình muốn sử dụng tài nguyên thì nó phải thông báo với semaphore.
 - Tại một thời điểm có thể có một hoặc nhiều thread có được khóa.
- Muốn sử dụng đối tượng semaphore, ta cần gọi hàm **sem_init()** để khởi tạo biến semaphore có kiểu cấu trúc **sem_t** như sau:

```
#include <semaphore>
int sem_init(sem_t* sem, int pshared, unsigned int value)
```

Trong đó,

sem_init() là lời gọi để bắt đầu một semaphore không tên. Nó mở một semaphore hiện có hoặc tạo một semaphore mới và mở nó cho hoạt động tiếp theo.

sem_t *sem: định nghĩa một biến con trỏ tên sem có kiểu cấu trúc là sem_t.

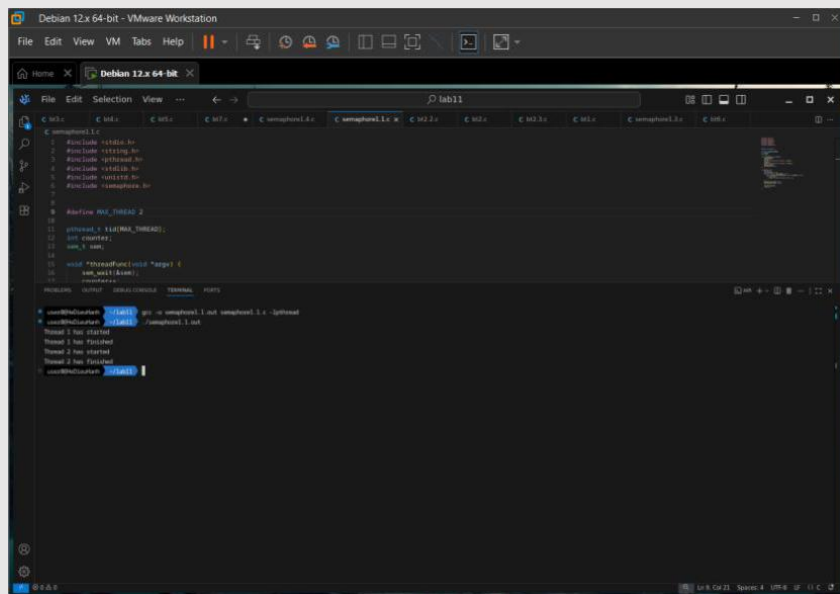
pshared: cho biết semaphore này dùng để liên lạc giữa tiến trình hay tiểu trình.

Nếu pshared = 0: tạo semaphore chia sẻ giữa các tiểu trình của một tiến trình.

Nếu $pshared = 1$ ($\neq 0$): tạo semaphore chia sẻ giữa các tiến trình. Trong trường hợp này thì semaphore phải được đặt trong phân đoạn bộ nhớ dùng chung được gắn với các tiến trình liên quan.

value: giá trị của semaphore được khởi tạo bởi **sem_init()**.
Xem thêm cách đồng bộ hóa tiến trình với semaphore, tìm hiểu **sem_wait()**, **sem_post()**

1.1 Sử dụng semaphore để đồng bộ các tiêu/ tiến trình trong các bài tập 1.1→1.4



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5 #include <semaphore.h>
6
7 #define NUM_THREADS 2
8
9 #define SEM_NAME "/tmp/semaphore1"
10
11 pthread_t t1, t2;
12 int counter;
13 sem_t sem;
14
15 void *thread_func(void *arg) {
16     sem_wait(&sem);
17     // Critical section
18     counter++;
19     printf("Thread %d: counter = %d\n", (int)arg, counter);
20     sem_post(&sem);
21     return NULL;
22 }
23
24 int main() {
25     if (sem_init(&sem, 0, 1) != 0) {
26         perror("sem_init");
27         return 1;
28     }
29     pthread_create(&t1, NULL, thread_func, (void *)1);
30     pthread_create(&t2, NULL, thread_func, (void *)2);
31     pthread_join(t1, NULL);
32     pthread_join(t2, NULL);
33     printf("Main: counter = %d\n", counter);
34     return 0;
35 }
```

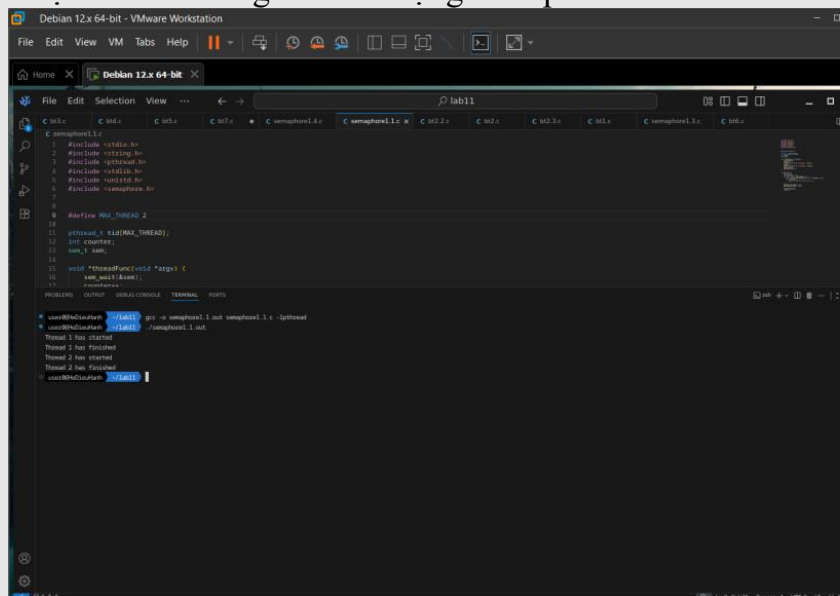
Terminal Output:

```
Thread 1 has started
Thread 1 has finished
Thread 2 has started
Thread 2 has finished
Main: counter = 2
```

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code chương trình sử dụng semaphore cho bài 1.1



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5 #include <semaphore.h>
6
7 #define NUM_THREADS 2
8
9 #define SEM_NAME "/tmp/semaphore1"
10
11 pthread_t t1, t2;
12 int counter;
13 sem_t sem;
14
15 void *thread_func(void *arg) {
16     sem_wait(&sem);
17     // Critical section
18     counter++;
19     printf("Thread %d: counter = %d\n", (int)arg, counter);
20     sem_post(&sem);
21     return NULL;
22 }
23
24 int main() {
25     if (sem_init(&sem, 0, 1) != 0) {
26         perror("sem_init");
27         return 1;
28     }
29     pthread_create(&t1, NULL, thread_func, (void *)1);
30     pthread_create(&t2, NULL, thread_func, (void *)2);
31     pthread_join(t1, NULL);
32     pthread_join(t2, NULL);
33     printf("Main: counter = %d\n", counter);
34     return 0;
35 }
```

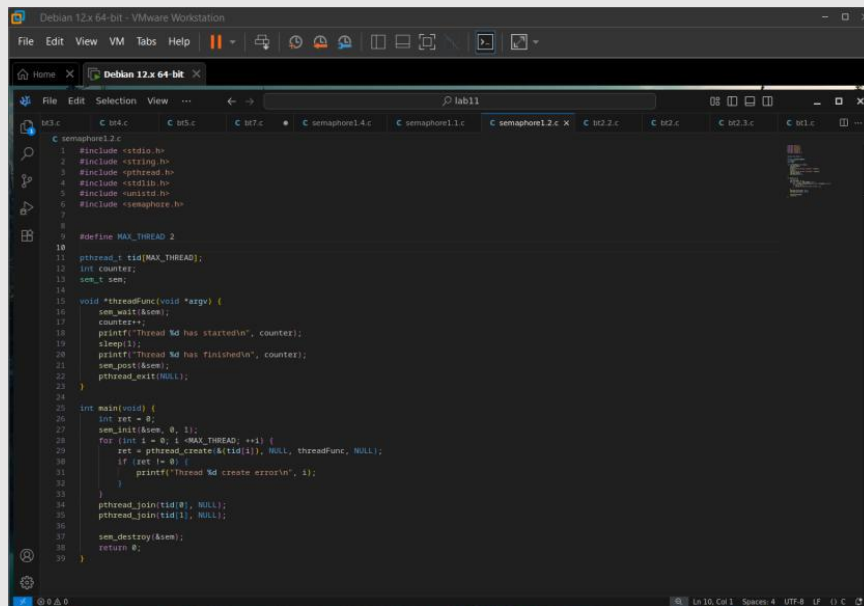
Terminal Output:

```
Thread 1 has started
Thread 1 has finished
Thread 2 has started
Thread 2 has finished
Main: counter = 2
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả của chương trình.

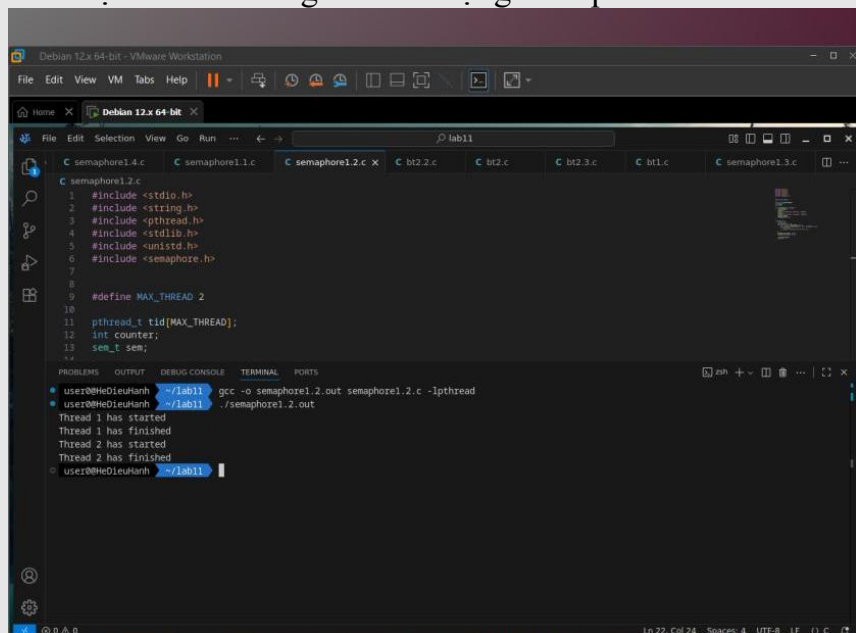


```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4 #include <unistd.h>
5 #include <semaphore.h>
6
7
8
9 #define MAX_THREAD 2
10
11 pthread_t tid[MAX_THREAD];
12 int counter;
13 sem_t sem;
14
15 void *threadFunc(void *argv) {
16     sem_wait(&sem);
17     counter++;
18     printf("Thread %d has started\n", counter);
19     sleep(1);
20     printf("Thread %d has finished\n", counter);
21     sem_post(&sem);
22     pthread_exit(NULL);
23 }
24
25 int main(void) {
26     int ret = 0;
27     sem_init(&sem, 0, 1);
28     for (int i = 0; i < MAX_THREAD; ++i) {
29         ret = pthread_create(&tid[i], NULL, threadFunc, NULL);
30         if (ret != 0) {
31             printf("Thread %d create error\n", i);
32         }
33     }
34     pthread_join(tid[0], NULL);
35     pthread_join(tid[1], NULL);
36     sem_destroy(&sem);
37     return 0;
38 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code chương trình sử dụng semaphore cho bài 1.2

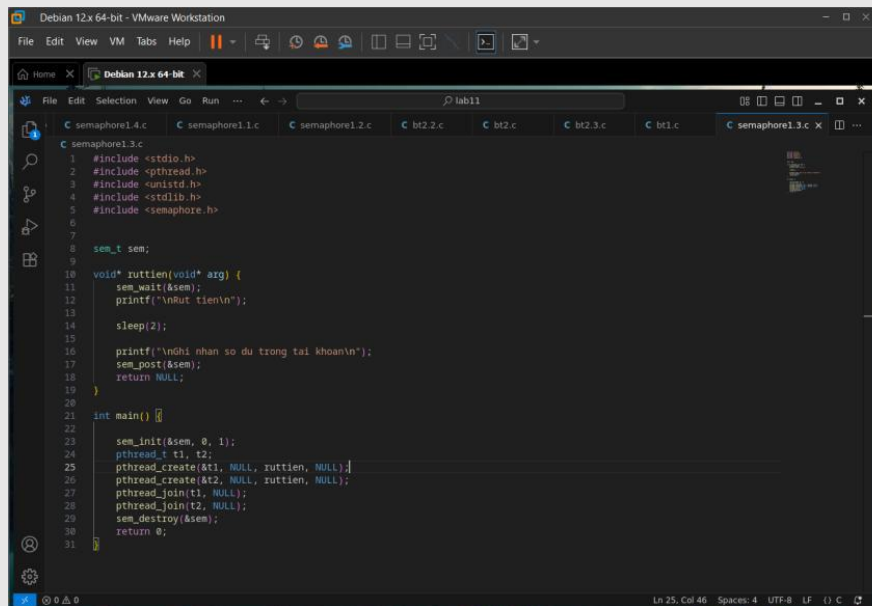


```
user@HeDieuKhanh: ~/lab11$ gcc -o semaphore1.2.out semaphore1.2.c -lpthread
user@HeDieuKhanh: ~/lab11$ ./semaphore1.2.out
Thread 1 has started
Thread 1 has finished
Thread 2 has started
Thread 2 has finished
user@HeDieuKhanh: ~/lab11$
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả của chương trình.

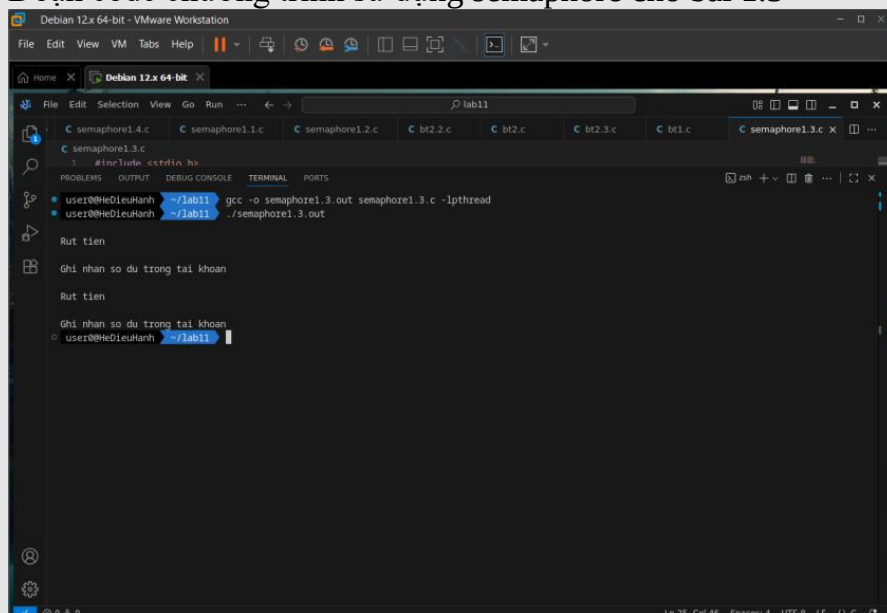


```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <semaphore.h>
6
7 sem_t sem;
8
9
10 void* ruttien(void* arg) {
11     sem_wait(&sem);
12     printf("\nRut tien\n");
13
14     sleep(2);
15
16     printf("\nghi nhan so du trong tai khoan\n");
17     sem_post(&sem);
18     return NULL;
19 }
20
21 int main() {
22
23     sem_init(&sem, 0, 1);
24     pthread_t t1, t2;
25     pthread_create(&t1, NULL, ruttien, NULL);
26     pthread_create(&t2, NULL, ruttien, NULL);
27     pthread_join(t1, NULL);
28     pthread_join(t2, NULL);
29     sem_destroy(&sem);
30     return 0;
31 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code chương trình sử dụng semaphore cho bài 1.3



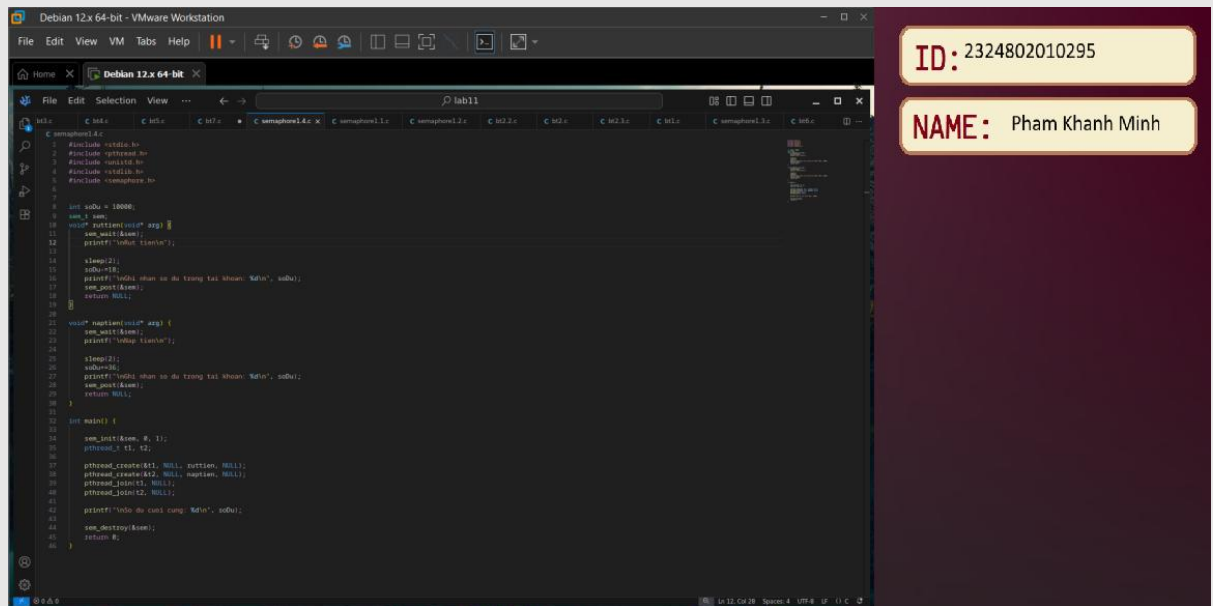
```
user@HeDieuHanh: ~/lab11$ gcc -o semaphore1.3.out semaphore1.3.c -lpthread
user@HeDieuHanh: ~/lab11$ ./semaphore1.3.out

Rut tien
ghi nhan so du trong tai khoan
Rut tien
ghi nhan so du trong tai khoan
user@HeDieuHanh: ~/lab11$
```

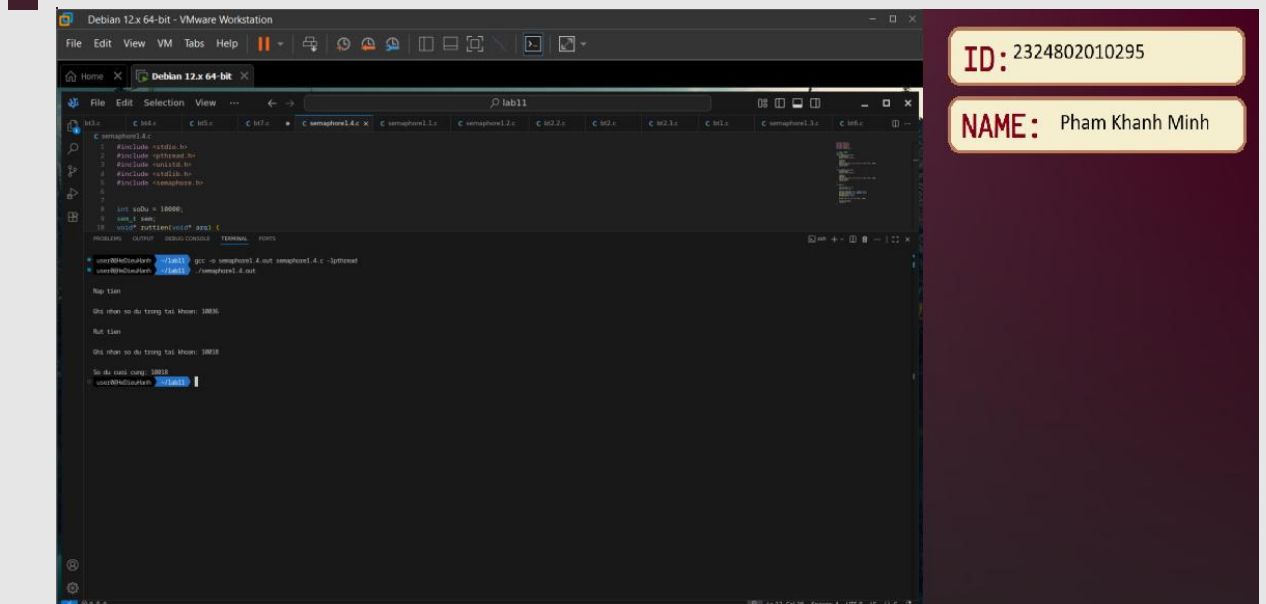
ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả của chương trình.



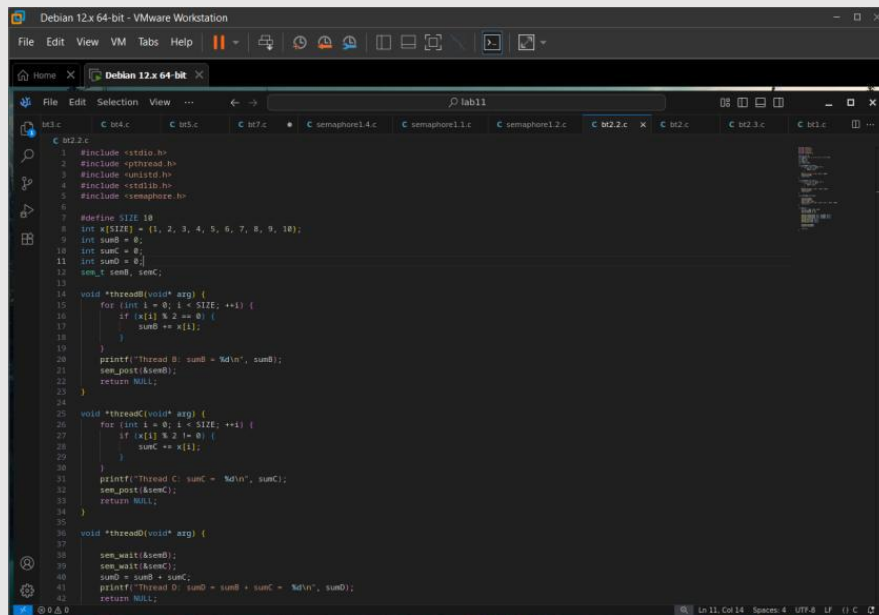
Đoạn code chương trình sử dụng semaphore cho bài 1.4



Màn hình trên là kết quả của chương trình.

1.2 Viết chương trình tạo mảng số nguyên có 10 phần tử `int x[10]`. Sử dụng cơ chế đồng bộ hóa tiến trình bằng mutex hoặc semaphore để viết code cho 3 thread B,C và D cùng thực hiện đồng thời các thao tác trên mảng x theo yêu cầu sau:

- B tính tổng (sumB) các phần tử chẵn trong mảng x.
- C tính tổng (sumC) các phần tử lẻ trong mảng x.
- D tính tổng các phần tử của mảng x dựa trên kết quả tính của B và C ($\text{sumD} = \text{sumB} + \text{sumC}$).

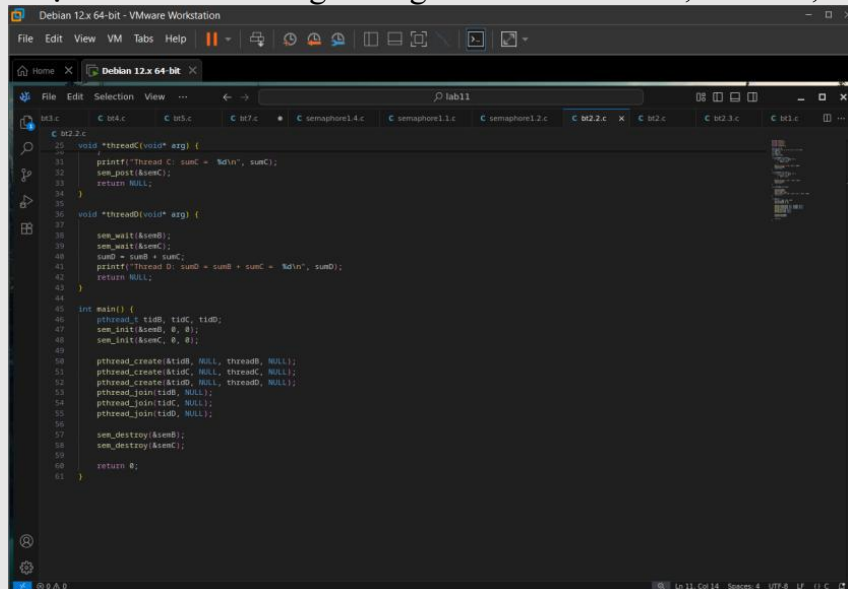


```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <semaphore.h>
6
7 #define SIZE 10
8 int x[SIZE] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
9 int sumB = 0;
10 int sumC = 0;
11 int sumD = 0;
12 sem_t semB, semC;
13
14 void *threadB(void* arg) {
15     for (int i = 0; i < SIZE; ++i) {
16         if (x[i] % 2 == 0) {
17             sumB += x[i];
18         }
19     }
20     printf("Thread B: sumB = %d\n", sumB);
21     sem_post(&semB);
22     return NULL;
23 }
24
25 void *threadC(void* arg) {
26     for (int i = 0; i < SIZE; ++i) {
27         if (x[i] % 2 != 0) {
28             sumC += x[i];
29         }
30     }
31     printf("Thread C: sumC = %d\n", sumC);
32     sem_post(&semC);
33     return NULL;
34 }
35
36 void *threadD(void* arg) {
37     sem_wait(&semB);
38     sem_wait(&semC);
39     sumD = sumB + sumC;
40     printf("Thread D: sumD = sumB + sumC = %d\n", sumD);
41     return NULL;
42 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Đoạn code của chương trình gồm 3 hàm threadB, threadC, threadD

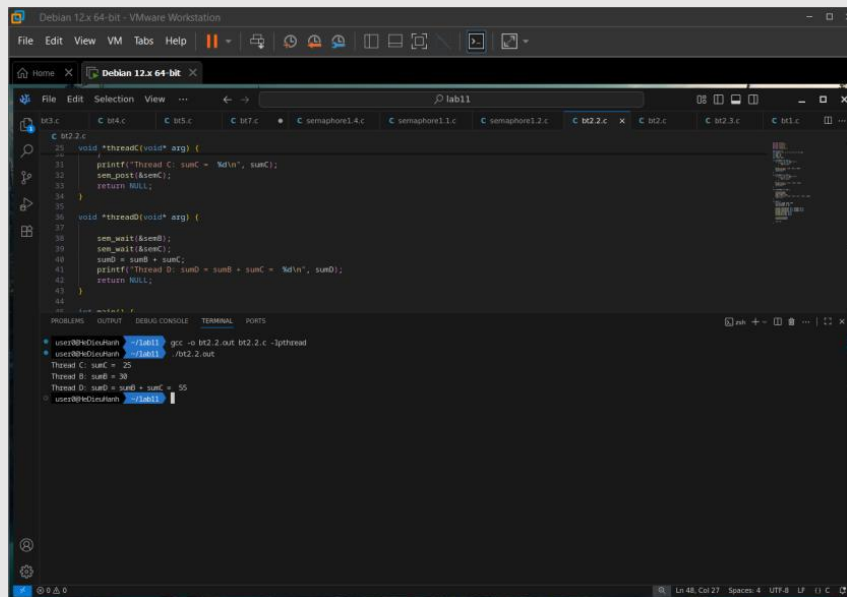


```
25 void *threadC(void* arg) {
26     printf("Thread C: sumC = %d\n", sumC);
27     sem_post(&semC);
28     return NULL;
29 }
30
31 void *threadD(void* arg) {
32     sem_wait(&semB);
33     sem_wait(&semC);
34     sumD = sumB + sumC;
35     printf("Thread D: sumD = sumB + sumC = %d\n", sumD);
36     return NULL;
37 }
38
39 int main() {
40     pthread_t tidB, tidC, tidD;
41     sem_init(&semB, 0, 0);
42     sem_init(&semC, 0, 0);
43
44     pthread_create(&tidB, NULL, threadB, NULL);
45     pthread_create(&tidC, NULL, threadC, NULL);
46     pthread_create(&tidD, NULL, threadD, NULL);
47     pthread_join(tidB, NULL);
48     pthread_join(tidC, NULL);
49     pthread_join(tidD, NULL);
50
51     sem_destroy(&semB);
52     sem_destroy(&semC);
53
54     return 0;
55 }
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là đoạn code hàm main của chương trình.



```
Debian 12.x 64-bit - VMware Workstation
File Edit View VM Tabs Help
Debian 12.x 64-bit
File Edit Selection View ...
lab11
C b13.c C b14.c C b15.c C b17.c C semaphore1.4.c C semaphore1.1.c C semaphore1.2.c C b12.2.c C b12.c C b12.3.c C b11.c
25 void *threadC(void* arg) {
26     /
27     printf("Thread C: sumC = %d\n", sumC);
28     sem_post(&semC);
29     return NULL;
30 }
31
32 void *threadD(void* arg) {
33     /
34     sem_wait(&semB);
35     sem_wait(&semC);
36     sumD = sumB + sumC;
37     printf("Thread D: sumD = sumB + sumC = %d\n", sumD);
38     return NULL;
39 }
40
41 int main() {
42     pthread_t t1, t2;
43     pthread_create(&t1, NULL, threadC, NULL);
44     pthread_create(&t2, NULL, threadD, NULL);
45     pthread_join(t1, NULL);
46     pthread_join(t2, NULL);
47     printf("Final sum: %d\n", sumD);
48     return 0;
49 }
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
user@kali:~/lab11$ gcc -o b12.2.out b12.2.c -lpthread
user@kali:~/lab11$ ./b12.2.out
Thread C: sumC = 25
Thread D: sumD = 30
Thread D: sumD = sumB + sumC = 55
user@kali:~/lab11$
```

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả chương trình trên tổng các số từ 1 – 10.

1.3 Một hãng sản xuất xe ô tô có các bộ phận hoạt động song song :

- Bộ phận sản xuất khung xe :

```
MakeChassis ( ) { // Sản xuất ra một khung xe
Produce_chassis ( ) ;}
```

- Bộ phận sản xuất bánh xe :

```
MakeTire ( ) { // Sản xuất ra một bánh xe
Produce_tire ( ) ;}
```

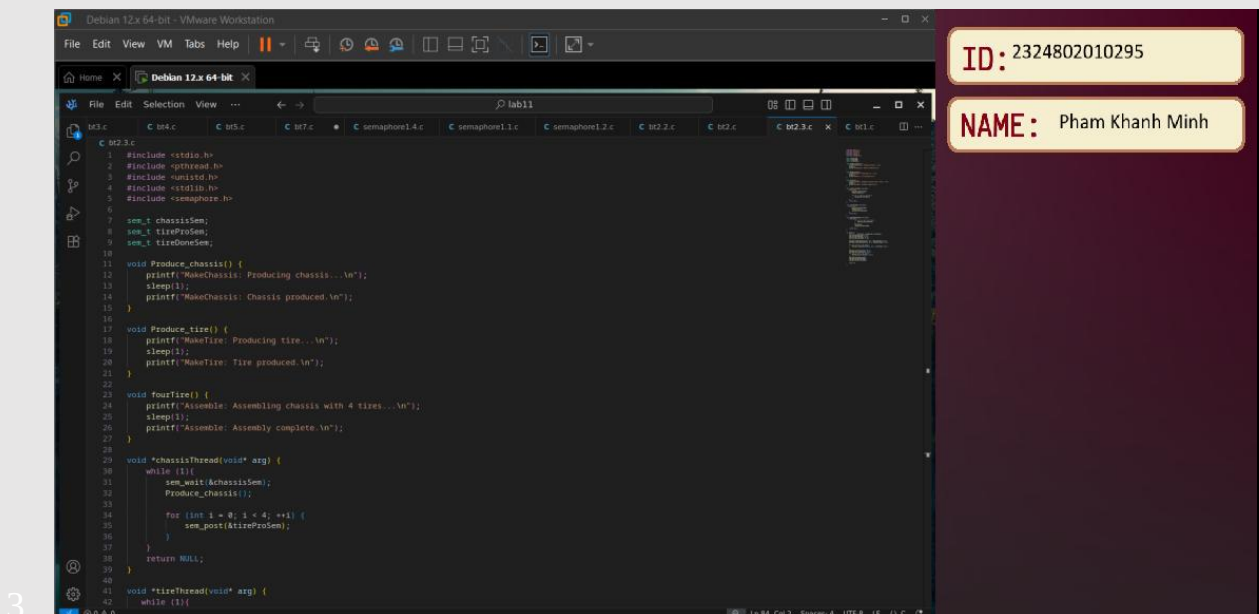
- Bộ phận lắp ráp : Sau khi có được 1 khung xe và 4 bánh xe thì tiến hành lắp ráp

4 bánh xe này vào khung xe :

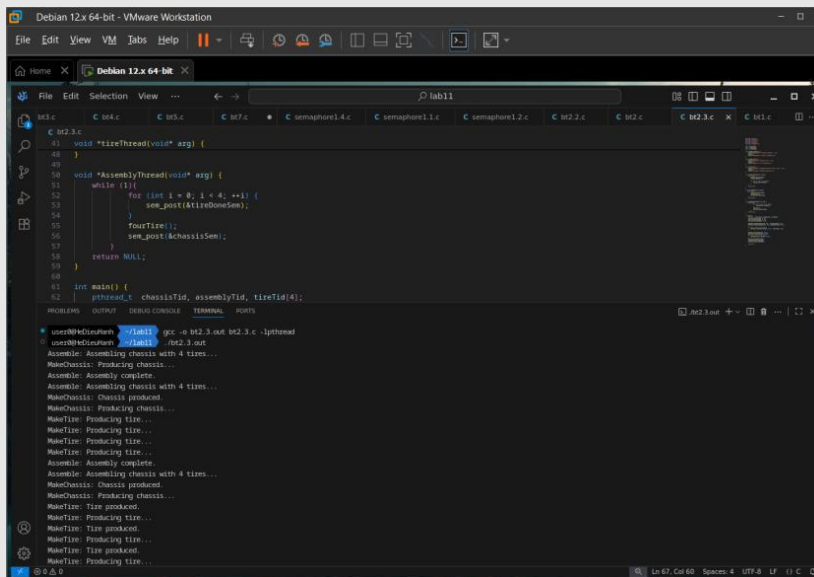
```
Assemble ( ) { // Gắn 4 bánh xe vào khung xe
Put_4_tires_to_chassis ( ) ;
}
```

Hãy đồng bộ hoạt động của các bộ phận trên thoả các nguyên tắc sau :

Tại mỗi thời điểm chỉ cho phép sản xuất ra 1 khung xe . Cần chờ có đủ 4 bánh xe để gắn vào khung xe hiện tại này trước khi sản xuất ra một khung xe mới .



Màn hình trên là code của chương trình.



```
1 void *tireThread(void* arg) {
2     while (1) {
3         for (int i = 0; i < 4; ++i) {
4             sem_post(&tireDoneSem);
5             FourTire();
6             sem_post(&chassisSem);
7         }
8         return NULL;
9     }
10 }
11
12 int main() {
13     pthread_t chassisTid, assemblyTid, tireTid[4];
14     pthread_create(&chassisTid, NULL, MakeChassis, NULL);
15     pthread_create(&assemblyTid, NULL, Assemble, NULL);
16     for (int i = 0; i < 4; ++i) {
17         pthread_create(&tireTid[i], NULL, tireThread, NULL);
18     }
19     pthread_join(chassisTid, NULL);
20     pthread_join(assemblyTid, NULL);
21     for (int i = 0; i < 4; ++i) {
22         pthread_join(tireTid[i], NULL);
23     }
24     return 0;
25 }
```

user@ubuntu:~/lab11\$ gcc -std=c11 -pthread lab11.c -o lab11.out

user@ubuntu:~/lab11\$./lab11.out

Assemble: Assembling chassis with 4 tires...

MakeChassis: Producing chassis...

Assemble: Assembly complete.

Assemble: Assembling chassis with 4 tires...

MakeChassis: Chassis produced.

MakeChassis: Producing chassis...

MakeTire: Producing tire...

MakeTire: Producing tire...

MakeTire: Producing tire...

MakeTire: Producing tire...

Assemble: Assembly complete.

Assemble: Assembling chassis with 4 tires...

MakeChassis: Chassis produced.

MakeChassis: Producing chassis...

MakeTire: Tire produced.

MakeTire: Producing tire...

MakeTire: Tire produced.

MakeTire: Producing tire...

MakeTire: Tire produced.

ID: 2324802010295

NAME: Pham Khanh Minh

Màn hình trên là kết quả của chương trình.